DATA STRUCTURES & ALGORITHMS · COMS-RW-20515

Final project: the three questions

Choose one · Report and code due Sunday 27 September, 23:59 · 40% of the module · Released 19 September

Why this has changed. Your proposals were good and most of them were too large for the time left. Rather than have eleven people each discover that separately, the three questions below are drawn from your own proposals, reduced to a size that can be completed in the time left, and the data is supplied. Your time from here is spent on the algorithms and the argument, not on finding a dataset. Pick one, and tell me which.

What every question requires

The three questions differ in subject. Each is marked on the same five requirements. Answer one question only. If a report answers more than one, only the question named on its first page is marked.

1. Two implementations: the method the question names, and the baseline it is compared against. Each question names its baseline under What you must compare.
2. A measured comparison of the two on the supplied data: numbers you produced, not an opinion.
3. One instance on which the two methods differ in the way the question is about, and one on which they do not. Both matter, and either may be an instance you construct yourself. In Question 1 both methods give the same number of rooms, so the difference is running time: an input needing many rooms, on which the list version is clearly slower, and one needing few, on which it is not. In Questions 2 and 3 the difference is the quality of the answer: how many areas are covered, or how much is raised.
4. The cost of both methods in Big-O, and the running times you actually measured. This applies to all three questions. Say where the analysis and the measurements disagree, and why. Read Measuring running time below before you start timing: most of the disagreements you will meet are properties of the machine rather than of your algorithm, and telling the two apart is the point of this requirement.
5. What your solution does not do. A limitation you identify yourself is not penalised. One the marker identifies is.

Deliverables: code with a README, and a report of five to ten pages with the AI Use Declaration cover sheet and prompt log. There is no viva: the report and code carry the full 40%. The rubric in the project brief applies to all three questions.

Measuring running time

Requirement 4 asks you to measure rather than assert, and then to account for any disagreement with your analysis. Four things will stop you reporting noise as a result.

Warm up before you measure. A just-in-time compiled runtime, which includes Java, C# and JavaScript, runs the first few hundred iterations of a loop slowly and then recompiles it. Run each implementation a few hundred times on a throwaway input before the timed run, or your first measurement will be of the compiler rather than of your algorithm.

Repeat, and report a median. A single timing catches whatever garbage collection happened to land inside it. Five runs and a median is enough here; say what you did.

Count operations as well as milliseconds. Put a counter on whatever your analysis is actually about (the comparison, the relaxation, or the table entry) and report it beside the clock. Operation counts are smooth and scale as the analysis predicts. Wall-clock

does not, because it also measures allocation, cache behaviour and the compiler. Where the two disagree, the counts are describing your algorithm and the clock is describing your machine, and saying precisely that is a better answer than either number alone.

Time the part that differs. If both implementations sort the input first, timing the whole program dilutes the very difference you are trying to show. Time the shared work separately, and say so.

This is measurement technique rather than the algorithmic core, so help with it is permitted under the AI policy in the project brief. Getting it wrong is still your problem.

Question 1: Rooms for a multi-track event

From Elera-Obari's proposal.

The problem. A conference runs 40 sessions in one day. Each has a start and an end time. Sessions that overlap need different rooms. A session ending at 10:00 does not overlap one starting at 10:00. How few rooms can run the whole programme, and which session goes in which room?

What you must build. Sort the sessions by start time, and keep a min-heap of the finish times of the rooms in use. For each session in turn, look at the room that becomes free earliest. If it is free at or before the session's start time, assign the session to it; otherwise open a new room. This is a greedy algorithm: each session is placed by a fixed rule and never moved. You need the room assignment for every session, not only the number of rooms.

What you must compare. The baseline is the same greedy algorithm with the heap replaced by a list of rooms, scanned for each session until a free room is found. Both give the same number of rooms; they differ in running time. Time both on the supplied data and on generated inputs of up to at least 5,000 sessions. The heap version is O(n log n) and the list version is O(n × r), where r is the number of rooms. Plot measured running time against n rather than asserting the difference. Note what that second bound means for the inputs you generate: it depends on r as well as n, and if you generate sessions the obvious way (a random start somewhere in the day and a random duration of an hour or two), you will get a handful of rooms however large n is, the list scan will be short, and you will measure almost no difference. Generate at least two families: one in which the number of rooms grows with n, and one in which it stays fixed however large n becomes. Say how you made each happen. Those two families are also the two instances that requirement 3 asks for. The generators are part of the algorithmic core of this question, not test-data scaffolding: you may learn with AI how to control the number of rooms, but the generators you submit must be your own.

What you must be able to justify. That your answer is optimal, not merely good. Compute, separately and by a different method, the maximum number of sessions running at the same moment. No schedule can use fewer rooms than that number: it is a lower bound. Check in code that the greedy room count equals the lower bound on every instance you generate, and explain in the report why the two must always be equal.

Your data. Q1_sessions_main.csv contains 40 sessions. Q1_sessions_selfcheck.csv is a self-check instance, small and with its answer given: it contains eight sessions and needs three rooms. Get that right before you run the main file.

If you finish early. Give each session a required room type, seminar or lab, and give each room a type. The greedy algorithm must now choose among rooms of the right type. Does the lower-bound argument still prove the greedy answer optimal? Answer carefully.

Question 2: A new bus route, and where to put the stops

From Umar's proposal.

The problem. There is no direct bus service from Curepipe to Pamplemousses. Model the road network as a weighted graph and find the fastest route. Then choose where to

place six stops, at any of the 18 junctions, so that as many residential areas as possible are covered. An area is covered if the shortest travel time between a stop and the area's junction is at most nine minutes; exactly nine minutes counts.

What you must build. Dijkstra's algorithm, for the fastest route and for the travel time from each junction to every area. Then a greedy rule for the stops: repeatedly add the junction that covers the most areas not yet covered, until there are six stops. Where two junctions tie, take the one whose name comes first in alphabetical order, so that every correct implementation gives the same greedy answer. The stops need not lie on the fastest route.

What you must compare. The baseline is exhaustive search: the greedy six against the best possible six. There are 18 junctions, so there are 18,564 ways to choose six. Enumerate them all and find the true maximum. Report both numbers and the gap between them, in areas.

What you must be able to justify. Two things. First, why the greedy rule is not guaranteed to find the best answer, and how close it came here. The gap on this network is narrow and turns partly on the tie-break, so it will not carry the argument by itself: construct a small instance of your own on which greedy loses by more, and use that to explain why the rule carries no guarantee. Second, what the exhaustive search cost, and how that cost grows as the network grows. Take care here, because the obvious answer is wrong. With the number of stops fixed at six, C(V, 6) is a polynomial in V of degree six, so doubling the network multiplies the work by about sixty-four; that is not intractability. Say what happens instead when the number of stops grows with the network (one stop for every three junctions, say), and which of those two situations the NP-hardness of this problem refers to. That is the connection to the Week 12 material, and it is the harder half of this question.

Your data. Q2_road_network.csv lists 25 roads with travel times in minutes; every road can be travelled in both directions. Q2_residential_areas.csv lists 18 areas and the junction at which each is located. Compute coverage yourself, from the same shortest paths.

If you finish early. Both of the following are optional, and neither is needed for full marks. Vary the number of stops from one to eight and plot coverage against the number of stops. At what point does an extra stop add little? Then use breadth-first search to find the route with the fewest roads, compare its travel time with the fastest route, and explain why minimising the number of roads and minimising travel time are different problems.

Question 3: Choosing what to take when you cannot take everything

From Princess's proposal, and the same type of problem as several others.

The problem. Twelve sponsors are listed in a fixed order, each offering a pledge. You may accept any of them, with one restriction: no two sponsors who are adjacent in the list, because neighbours are competitors and will not both sign. The order cannot be changed. What is the largest total you can raise, and from which sponsors?

What you must build. A bottom-up dynamic-programming table with one entry per position, filled from the smallest subproblem upwards, as in Week 11. Before writing code, answer in writing the five questions from the Week 11 lecture: (1) what one subproblem is; (2) the recurrence, that is, how a subproblem is computed from smaller ones; (3) the base cases; (4) the order in which the table is filled, so that every entry it depends on is already computed; and (5) where in the table the final answer is found. Then trace back through the table to identify which sponsors were accepted, not only the total.

What you must compare. The baseline is a greedy rule: repeatedly accept the largest remaining pledge whose neighbours have not been accepted. Once a sponsor's neighbour

has been accepted, that sponsor is out of the running permanently, not merely passed over for this round. Where two pledges are equal, take the one earlier in the list, so that every correct implementation gives the same greedy answer. Run both on the supplied list. The greedy rule gives a smaller total on this instance. Say by how much, and construct a list on which it gives the same total as the table.

What you must be able to justify. Why the table is correct and the greedy rule is not, argued in words as in the Week 11 lecture and whiteboard session. Take the last decision the best answer makes. What it leaves behind must itself be solved as well as possible, because otherwise a better solution to that smaller problem could be substituted, beating the answer you started with. This property is called optimal substructure. The greedy rule has no such guarantee.

Your data. Q3_sequence_main.csv contains twelve pledges. Q3_sequence_selfcheck.csv contains five pledges, and the answer is 17. If you attempt the extension, Q3_items_main.csv and Q3_items_selfcheck.csv each carry a capacity column, repeated on every row, so that the capacity is in the data and not only in this sheet.

If you finish early. This is the extension. Like the extensions to the other two questions it is optional, and a complete answer to the core question can reach any band without it. Drop the adjacency restriction. Give every item a weight as well as a value, and set a capacity: the total weight of the items taken may not exceed it. This is the 0/1 knapsack problem. One index no longer describes a subproblem: it needs two, the item you have reached and the capacity remaining. Q3_items_main.csv has eight items and a capacity of 21. Two greedy baselines are worth beating: highest value first, and highest value per unit of weight first. Q3_items_selfcheck.csv, with a capacity of 9, gives 27.

Choosing

Question 1 is the smallest to build and the hardest to write up: its marks depend almost entirely on the optimality argument and the measurement rather than on the code, so choose it if you are comfortable arguing in prose. Question 2 is the largest build by some margin (a graph, Dijkstra with route recovery, shortest paths from every junction, the coverage model, and the enumeration), and is the best choice if graphs are your strongest topic; its two extensions are optional and are not needed for full marks. Question 3 is the only one that uses the Week 11 material, which Quiz 5 also examined; its core is the most contained on the sheet and its optional extension is the hardest single piece of work on it. All three are marked on the same rubric, and a complete answer to the core of any of them can reach any band.

Tell me by email which you have chosen, as soon as you have decided. If more than half the class converges on one, I will ask some of you to move, and I will ask the people who chose it last.

Dates

When	What
Saturday 19 September	Questions, data, and revised brief released. Choose one, and tell me which by email.
Sunday 27 September, 23:59	Report and code due.

The data files are linked at the bottom of the Final project assignment page on Canvas. The self-check instances are small and their answers are given: get those right first. The main instances are yours to answer.